RELATORIO DE USO DE IA

PROMPT EXATO ENVIADO:
Como faço a parte de Auth do meu projeto em Node com Express, Knex e JWT? Preciso receber email e senha, buscar no banco e devolver um token.

RESPOSTA EXATA DA IA:
Você pode fazer assim no service de autenticação:

```
const jwt = require("jsonwebtoken");
const db = require("../database/connection");

async function login(email, senha) {
  const usuario = await db("usuarios")
    .where({ email, senha })
    .first();

  if (!usuario) {
    throw new Error("Email ou senha inválidos");
  }

  const token = jwt.sign(
    { id: usuario.id, email: usuario.email },
    process.env.JWT_SECRET || "segredo",
    { expiresIn: "1d" }
  );

  return {
    usuario: {
      id: usuario.id,
      nome: usuario.nome,
      email: usuario.email
    },
    token
  };
}

module.exports = {
  login
};
```

E no controller:

```
const authService = require("../services/authService");

async function login(req, res) {
  try {
    const { email, senha } = req.body;

    const resultado = await authService.login(email, senha);

    return res.json(resultado);
  } catch (error) {
    return res.status(401).json({ erro: error.message });
  }
}

module.exports = {
```

```
  login
};
```

O QUE EU APRENDI ATRAVES DISSO:
Aprendi que a parte de login não deve ficar toda dentro da rota. O controller recebe os dados e o service faz a busca no banco e gera o token. Também entendi que o JWT guarda informações do usuário para ele acessar outras rotas depois.

PROMPT EXATO ENVIADO:
Como faço um middleware para proteger minhas rotas de palpites usando o token que vem no Authorization?

RESPOSTA EXATA DA IA:
Você pode criar um middleware assim:

```
const jwt = require("jsonwebtoken");

function authMiddleware(req, res, next) {
  const authHeader = req.headers.authorization;

  if (!authHeader) {
    return res.status(401).json({ erro: "Token não enviado" });
  }

  const partes = authHeader.split(" ");

  if (partes.length !== 2) {
    return res.status(401).json({ erro: "Token inválido" });
  }

  const [tipo, token] = partes;

  if (tipo !== "Bearer") {
    return res.status(401).json({ erro: "Formato do token inválido" });
  }

  try {
    const decoded = jwt.verify(
      token,
      process.env.JWT_SECRET || "segredo"
    );

    req.usuarioId = decoded.id;

    return next();
  } catch (error) {
    return res.status(401).json({ erro: "Token inválido ou expirado" });
  }
}

module.exports = authMiddleware;
```

Depois é só usar nas rotas:

```
const express = require("express");
const router = express.Router();
```

```
const authMiddleware = require("../middlewares/authMiddleware");
const palpitesController = require("../controllers/palpitesController");

router.get("/palpites", authMiddleware, palpitesController.listar);
router.post("/palpites", authMiddleware, palpitesController.criar);
router.put("/palpites/:id", authMiddleware, palpitesController.atualizar);
router.delete("/palpites/:id", authMiddleware, palpitesController.deletar);

module.exports = router;
```

O QUE EU APRENDI ATRAVES DISSO:
Aprendi como bloquear uma rota para só funcionar se o usuário estiver logado. Também entendi que o token vem no header Authorization e que o middleware usa o next() para deixar a rota continuar quando está tudo certo.

PROMPT EXATO ENVIADO:
Estou tentando cadastrar um palpite, mas preciso validar se os gols não são negativos antes de salvar no banco. Como ficaria esse código com Knex?

RESPOSTA EXATA DA IA:
Você pode fazer essa validação no service antes do insert:

```
const db = require("../database/connection");

async function criarPalpite(usuarioId, dados) {
  const { jogo, gols_a, gols_b, data_jogo } = dados;

  if (!jogo || gols_a === undefined || gols_b === undefined || !data_jogo) {
    throw new Error("Preencha todos os campos obrigatórios");
  }

  if (gols_a < 0 || gols_b < 0) {
    throw new Error("Os gols não podem ser negativos");
  }

  const [id] = await db("palpites").insert({
    usuario_id: usuarioId,
    jogo,
    gols_a,
    gols_b,
    data_jogo
  });

  const palpiteCriado = await db("palpites")
    .where({ id })
    .first();

  return palpiteCriado;
}

module.exports = {
  criarPalpite
};
```

No controller ficaria assim:

```
const palpitesService = require("../services/palpitesService");
```

```
async function criar(req, res) {
  try {
    const palpite = await palpitesService.criarPalpite(
      req.usuarioId,
      req.body
    );

    return res.status(201).json(palpite);
  } catch (error) {
    return res.status(400).json({ erro: error.message });
  }
}

module.exports = {
  criar
};
```

O QUE EU APRENDI ATRAVES DISSO:
Aprendi que antes de salvar qualquer coisa no banco é melhor validar os dados. Nesse caso, se os gols forem negativos, o sistema já retorna erro e não deixa cadastrar um palpite errado.

PROMPT EXATO ENVIADO:
Como faço para minha rota de criar palpite buscar informações externas com axios usando Promise.all antes de salvar no banco?

RESPOSTA EXATA DA IA:
Você pode usar Promise.all para fazer as duas requisições ao mesmo tempo:

```
const axios = require("axios");
const db = require("../database/connection");

async function criarPalpite(usuarioId, dados) {
  const { jogo, gols_a, gols_b, data_jogo } = dados;

  if (gols_a < 0 || gols_b < 0) {
    throw new Error("Os gols não podem ser negativos");
  }

  const ano = new Date(data_jogo).getFullYear();

  const [respostaDolar, respostaFeriados] = await Promise.all([
    axios.get("https://economia.awesomeapi.com.br/json/last/USD-BRL"),
    axios.get(`https://brasilapi.com.br/api/feriados/v1/${ano}`)
  ]);

  const dolar = Number(respostaDolar.data.USDBRL.bid);

  const feriados = respostaFeriados.data;

  const ehFeriado = feriados.some((feriado) => {
    return feriado.date === data_jogo;
  });

  const [id] = await db("palpites").insert({
    usuario_id: usuarioId,
    jogo,
```

```
    gols_a,
    gols_b,
    data_jogo,
    dolar_no_dia: dolar,
    dia_de_feriado: ehFeriado
  });

  const palpite = await db("palpites")
    .where({ id })
    .first();

  return palpite;
}

module.exports = {
  criarPalpite
};
```

O QUE EU APRENDI ATRAVES DISSO:
Aprendi que o Promise.all serve para rodar duas buscas ao mesmo tempo, em vez de fazer uma e depois a outra. Também entendi como usar dados vindos de API externa e salvar junto com o palpite no banco.

PROMPT EXATO ENVIADO:
Meu PUT de palpite está alterando qualquer palpite pelo id. Como faço para permitir alterar só se o palpite for do usuário logado?

RESPOSTA EXATA DA IA:
Você pode buscar o palpite usando o id e o usuario_id juntos. Assim, se o palpite não pertencer ao usuário logado, ele não será encontrado.

```
const db = require("../database/connection");

async function atualizarPalpite(id, usuarioId, dados) {
  const palpite = await db("palpites")
    .where({ id, usuario_id: usuarioId })
    .first();

  if (!palpite) {
    throw new Error("Palpite não encontrado para este usuário");
  }

  const { jogo, gols_a, gols_b, data_jogo } = dados;

  if (gols_a < 0 || gols_b < 0) {
    throw new Error("Os gols não podem ser negativos");
  }

  await db("palpites")
    .where({ id, usuario_id: usuarioId })
    .update({
      jogo,
      gols_a,
      gols_b,
      data_jogo
    });

  const palpiteAtualizado = await db("palpites")
```

```
    .where({ id, usuario_id: usuarioId })
    .first();

  return palpiteAtualizado;
}

module.exports = {
  atualizarPalpite
};
```

No controller:

```
async function atualizar(req, res) {
  try {
    const { id } = req.params;

    const palpite = await palpitesService.atualizarPalpite(
      id,
      req.usuarioId,
      req.body
    );

    return res.json(palpite);
  } catch (error) {
    return res.status(404).json({ erro: error.message });
  }
}
```

O QUE EU APRENDI ATRAVES DISSO:
Aprendi que não basta procurar o palpite só pelo id, porque outro usuário poderia alterar um palpite que não é dele. O correto é sempre usar o id do palpite junto com o id do usuário logado.